The operator workflow, step by step

This book follows the seven steps of the EKOS Lakehouse operator workflow on iDempiere 12 — an open-source ERP of 903 tables on Oracle XE, with its Java code, migration scripts and documentation — as an operator does them, in the EKOS web console. Each chapter shows a step's screen, what the operator clicks, the run log the console kept of it, what happens behind the screen, what the operator gets, and the caveats.

The screenshots, the run logs and every number are generated: a script walks the console's pages and the book is rendered from what it captured, so the book cannot drift from the product. Only text like this paragraph is written by a person.

The state is honest: on this testbed the review (step 3) is not fully signed — 195 code ↔ database conflicts and one identity match wait for a person — so the bar shows steps 4–7 as blocked behind it, although each was built and run (the gate is the product's, and it was not bypassed). Steps 3 and 6 quote no run log: the sign-offs and the subset load ran before EKOS recorded a log for them (RFC 0218).

The steps

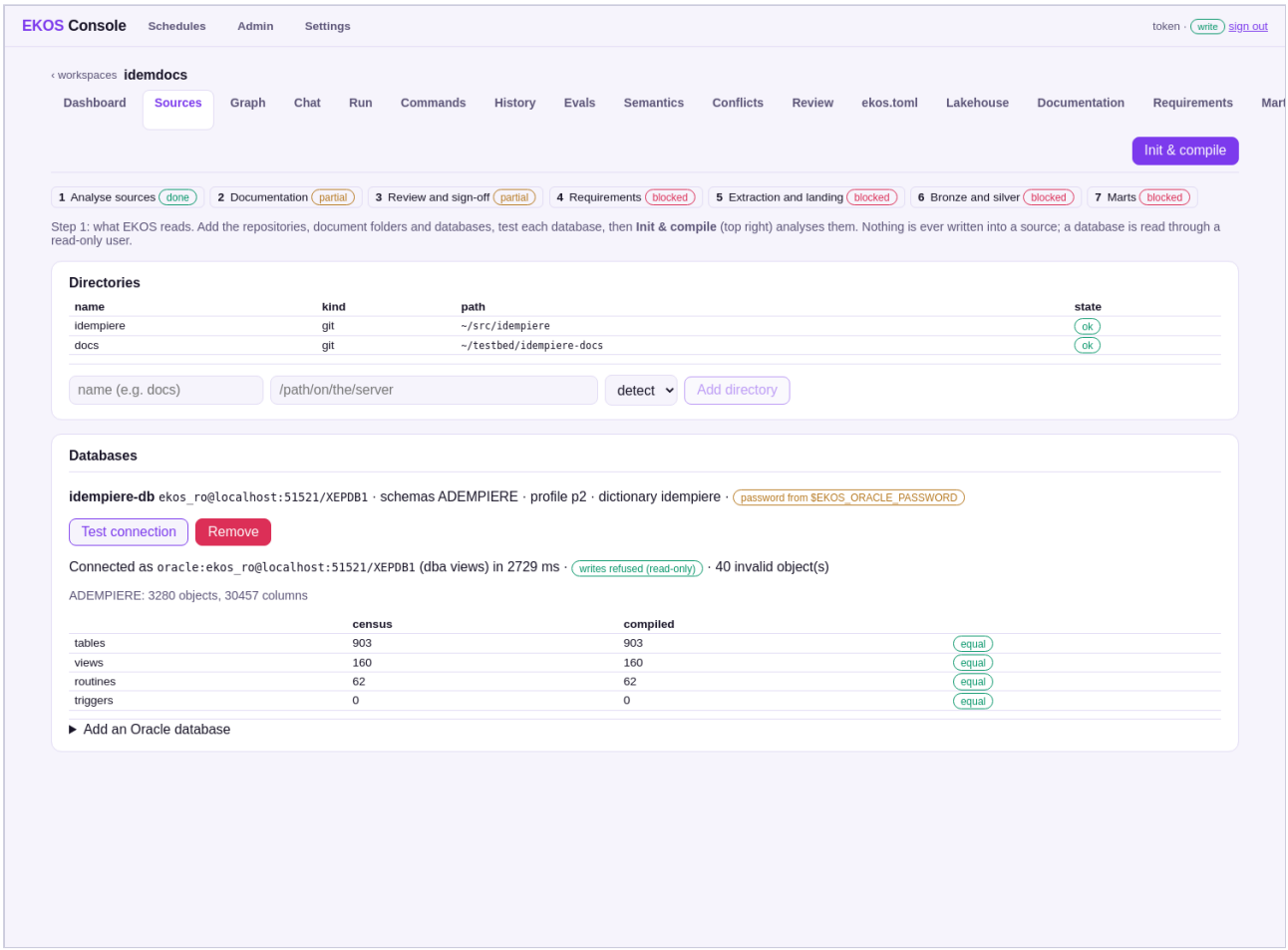
Step	State	Model calls in the quoted runs
1 Analyse sources	done	0
2 Documentation	partial	0
3 Review and sign-off	partial	—
4 Requirements	blocked	0
5 Extraction and landing	blocked	0
6 Bronze and silver	blocked	—
7 Marts	blocked	0

Captured 2026-10-26T17:22:29+00:00 from the console at workspace `idemdocs` (ekos-lakehouse 0.1.1 (EKOS 1.1.0)). Screenshots, run logs and numbers are regenerated on every capture; only the marked sections are written by a person. Local paths and account names are shortened in the text, the logs and the screenshots.

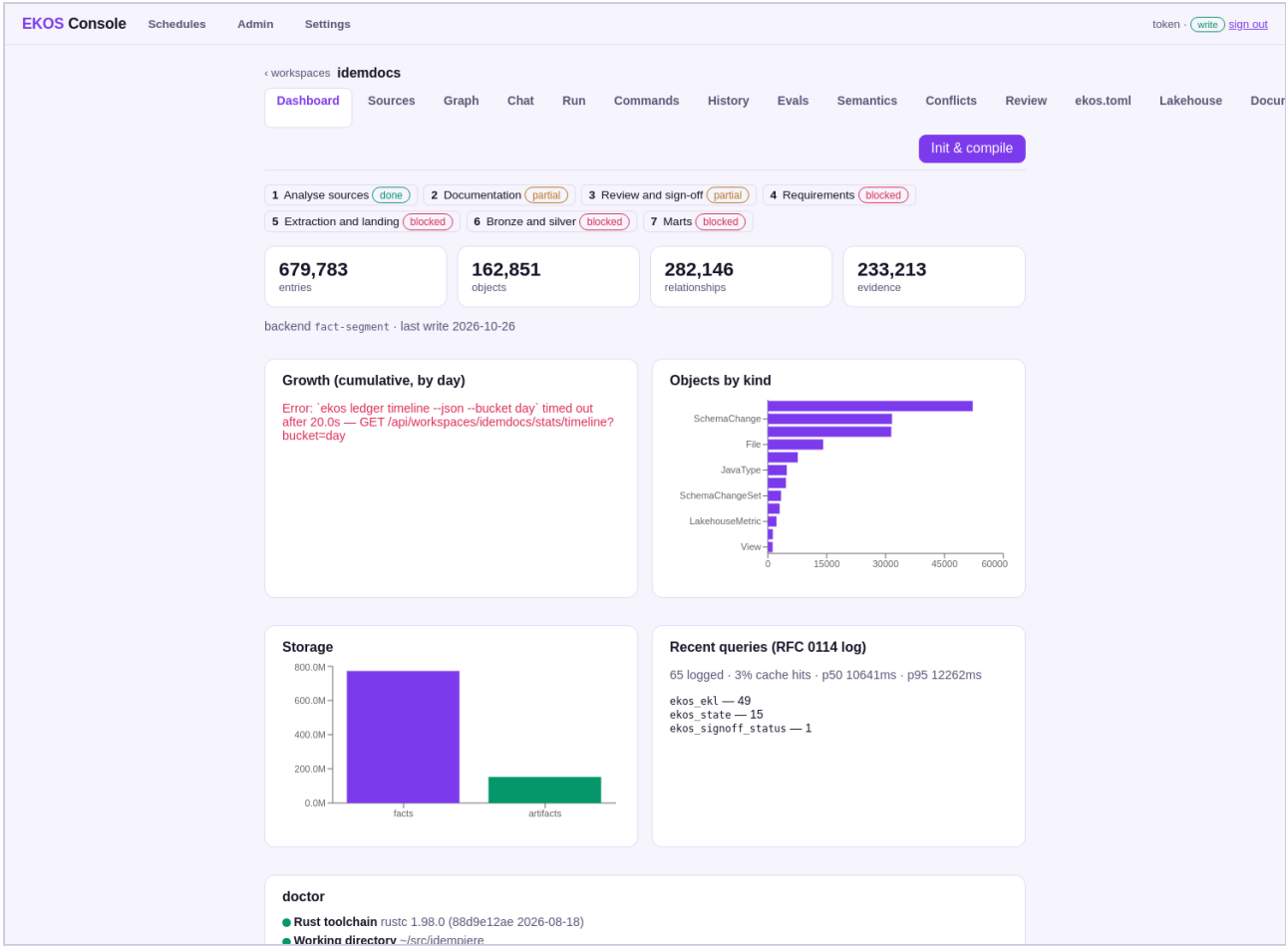
Step 1 — Analyse sources

On the step bar: done — fresh — ledger reflects the source as of 2026-10-25 16:56 UTC (git 3355eb5)

What the operator sees



The Sources page: the database, the code and the documents of the project.



The workspace: the step bar and what the ledger holds.

What they click

On **Sources** the operator adds the database (its connection; the password goes to the keychain), the code repositories and the documentation, tests the connection, and clicks **Analyse**.

What happens behind it

The connectors read the sources as they are: files from the repositories, the live catalog from Oracle (inside a read-only transaction, with a read-only account). Deterministic analyzers turn them into one ledger: Java types and the application model, SQL and PL/SQL, migration scripts as schema history, documents with their tickets, and the database's own tables, routines, keys and constraints. The census the catalog reports is reconciled against what was compiled. No language model reads any of it.

The run log

`compile` — succeeded, from a terminal, 2026-10-25T16:53:16+00:00; model calls: 0.

```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "compile", "t_ms": 0}
{"stage": "pass", "msg": "slow", "ms": 14022, "pass": "semantic-compiler", "cmd": "compile", "t_ms":
```

```
19990}
{"stage": "pass", "msg": "passes", "cached": 0, "failed": 0, "ms": 14032, "ran": 1, "cmd": "compile",
"t_ms": 19994}
{"stage": "semantic.compile", "msg": "model written", "objects": 144073, "relationships": 218326, "cmd":
"compile", "t_ms": 21443}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "compile", "t_ms":
33593}
{"stage": "run", "msg": "finished", "ok": true, "ms": 33593, "warnings": 134048, "cmd": "compile",
"t_ms": 33593}
```


`commit` — succeeded, from a terminal, 2026-10-25T16:53:50+00:00; model calls: 0.


```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "commit", "t_ms": 0}
{"stage": "ledger.commit", "msg": "facts written", "conflicts": 446, "evidence": 204689, "objects": 0,
"objects_skipped": 144073, "relationships": 0, "cmd": "commit", "t_ms": 187515}
{"stage": "write.behaviour", "msg": "tables classified", "tables": 903, "update": {"no_updated_column":
14, "po_only": 737, "raw_sql_sets_updated": 8, "raw_sql_skips_updated": 144}, "delete":
{"deleted_by_app": 107, "deleted_by_cascade": 298, "deleted_by_users": 754, "never_deleted_in_code": 9,
"purged": 12, "soft_delete_only": 57}, "change_log": {"not_recorded": 180, "recorded": 713, "unknown":
10}, "dictionary": true, "cmd": "commit", "t_ms": 187515}
{"stage": "tickets.link", "msg": "ticket links", "tickets": 4606, "links": {"JavaMethod": 272,
"JavaType": 154, "SchemaChangeSet": 3074, "Section": 1743}, "cited_without_commits": 55, "cmd":
"commit", "t_ms": 187515}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "commit", "t_ms":
188666}
{"stage": "run", "msg": "finished", "ok": true, "ms": 188666, "warnings": 0, "cmd": "commit", "t_ms":
188666}
```


`database test` — succeeded, from a terminal, 2026-10-26T14:11:42+00:00; model calls: 0.


```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "database test", "t_ms": 0}
{"stage": "oracle.catalog", "msg": "read DBA_OBJECTS", "ms": 35, "owner": "ADEMPIERE", "query":
"objects", "rows": 3280, "cmd": "database test", "t_ms": 84}
{"stage": "oracle.catalog", "msg": "read DBA_TABLES", "ms": 28, "owner": "ADEMPIERE", "query": "tables",
"rows": 903, "cmd": "database test", "t_ms": 112}
{"stage": "oracle.catalog", "msg": "read DBA_TAB_COLUMNS", "ms": 280, "owner": "ADEMPIERE", "query":
"columns", "rows": 30457, "cmd": "database test", "t_ms": 393}
{"stage": "oracle.catalog", "msg": "read DBA_CONSTRAINTS", "ms": 181, "owner": "ADEMPIERE", "query":
"constraints", "rows": 18444, "cmd": "database test", "t_ms": 575}
{"stage": "oracle.catalog", "msg": "read DBA_CONS_COLUMNS", "ms": 88, "owner": "ADEMPIERE", "query":
"constraint_columns", "rows": 18697, "cmd": "database test", "t_ms": 663}
{"stage": "oracle.catalog", "msg": "read DBA_INDEXES", "ms": 31, "owner": "ADEMPIERE", "query":
"indexes", "rows": 2109, "cmd": "database test", "t_ms": 695}
{"stage": "oracle.catalog", "msg": "read DBA_IND_COLUMNS", "ms": 30, "owner": "ADEMPIERE", "query":
"index_columns", "rows": 2624, "cmd": "database test", "t_ms": 726}
{"stage": "oracle.catalog", "msg": "read DBA_VIEWS", "ms": 10, "owner": "ADEMPIERE", "query": "views",
"rows": 160, "cmd": "database test", "t_ms": 738}
{"stage": "oracle.catalog", "msg": "read DBA_MVIEWS", "ms": 0, "owner": "ADEMPIERE", "query": "mviews",
"rows": 0, "cmd": "database test", "t_ms": 738}
{"stage": "oracle.catalog", "msg": "read DBA_SEQUENCES", "ms": 1, "owner": "ADEMPIERE", "query":
"sequences", "rows": 0, "cmd": "database test", "t_ms": 739}
{"stage": "oracle.catalog", "msg": "read DBA_SOURCE", "ms": 26, "owner": "ADEMPIERE", "query": "source",
"rows": 4753, "cmd": "database test", "t_ms": 766}
{"stage": "oracle.catalog", "msg": "read DBA_TRIGGERS", "ms": 1, "owner": "ADEMPIERE", "query":
"triggers", "rows": 0, "cmd": "database test", "t_ms": 767}
{"stage": "oracle.catalog", "msg": "read DBA_SYNONYMS", "ms": 1, "owner": "ADEMPIERE", "query":
"synonyms", "rows": 0, "cmd": "database test", "t_ms": 769}
{"stage": "oracle.catalog", "msg": "read DBA_TAB_COMMENTS", "ms": 2, "owner": "ADEMPIERE", "query":
"table_comments", "rows": 42, "cmd": "database test", "t_ms": 771}
{"stage": "oracle.catalog", "msg": "read DBA_COL_COMMENTS", "ms": 18, "owner": "ADEMPIERE", "query":
"column_comments", "rows": 111, "cmd": "database test", "t_ms": 790}
{"stage": "oracle.catalog", "msg": "read DBA_TAB_COL_STATISTICS", "ms": 162, "owner": "ADEMPIERE",
"query": "column_statistics", "rows": 18527, "cmd": "database test", "t_ms": 952}
{"stage": "oracle.catalog", "msg": "reconciled against DBA_OBJECTS", "kinds": 13, "mismatches": 0,
"objects": 10613, "owner": "ADEMPIERE", "cmd": "database test", "t_ms": 1095}
```

{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "database test", "t_ms": 13075}
{"stage": "run", "msg": "finished", "ok": true, "ms": 13075, "warnings": 0, "cmd": "database test", "t_ms": 13075}

`database test` — succeeded, from a terminal, 2026-10-26T17:20:47+00:00; model calls: 0.

{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "database test", "t_ms": 0}
{"stage": "oracle.catalog", "msg": "read DBA_OBJECTS", "ms": 385, "owner": "ADEMPIERE", "query": "objects", "rows": 3280, "cmd": "database test", "t_ms": 634}
{"stage": "oracle.catalog", "msg": "read DBA_TABLES", "ms": 106, "owner": "ADEMPIERE", "query": "tables", "rows": 903, "cmd": "database test", "t_ms": 741}
{"stage": "oracle.catalog", "msg": "read DBA_TAB_COLUMNS", "ms": 350, "owner": "ADEMPIERE", "query": "columns", "rows": 30457, "cmd": "database test", "t_ms": 1091}
{"stage": "oracle.catalog", "msg": "read DBA_CONSTRAINTS", "ms": 382, "owner": "ADEMPIERE", "query": "constraints", "rows": 18444, "cmd": "database test", "t_ms": 1473}
{"stage": "oracle.catalog", "msg": "read DBA_CONS_COLUMNS", "ms": 133, "owner": "ADEMPIERE", "query": "constraint_columns", "rows": 18697, "cmd": "database test", "t_ms": 1607}
{"stage": "oracle.catalog", "msg": "read DBA_INDEXES", "ms": 222, "owner": "ADEMPIERE", "query": "indexes", "rows": 2109, "cmd": "database test", "t_ms": 1829}
{"stage": "oracle.catalog", "msg": "read DBA_IND_COLUMNS", "ms": 55, "owner": "ADEMPIERE", "query": "index_columns", "rows": 2624, "cmd": "database test", "t_ms": 1885}
{"stage": "oracle.catalog", "msg": "read DBA_VIEWS", "ms": 58, "owner": "ADEMPIERE", "query": "views", "rows": 160, "cmd": "database test", "t_ms": 1943}
{"stage": "oracle.catalog", "msg": "read DBA_MVIEWS", "ms": 6, "owner": "ADEMPIERE", "query": "mviews", "rows": 0, "cmd": "database test", "t_ms": 1950}
{"stage": "oracle.catalog", "msg": "read DBA_SEQUENCES", "ms": 5, "owner": "ADEMPIERE", "query": "sequences", "rows": 0, "cmd": "database test", "t_ms": 1956}
{"stage": "oracle.catalog", "msg": "read DBA_SOURCE", "ms": 54, "owner": "ADEMPIERE", "query": "source", "rows": 4753, "cmd": "database test", "t_ms": 2010}
{"stage": "oracle.catalog", "msg": "read DBA_TRIGGERS", "ms": 22, "owner": "ADEMPIERE", "query": "triggers", "rows": 0, "cmd": "database test", "t_ms": 2032}
{"stage": "oracle.catalog", "msg": "read DBA_SYNONYMS", "ms": 12, "owner": "ADEMPIERE", "query": "synonyms", "rows": 0, "cmd": "database test", "t_ms": 2045}
{"stage": "oracle.catalog", "msg": "read DBA_TAB_COMMENTS", "ms": 20, "owner": "ADEMPIERE", "query": "table_comments", "rows": 42, "cmd": "database test", "t_ms": 2065}
{"stage": "oracle.catalog", "msg": "read DBA_COL_COMMENTS", "ms": 35, "owner": "ADEMPIERE", "query": "column_comments", "rows": 111, "cmd": "database test", "t_ms": 2101}
{"stage": "oracle.catalog", "msg": "read DBA_TAB_COL_STATISTICS", "ms": 359, "owner": "ADEMPIERE", "query": "column_statistics", "rows": 18527, "cmd": "database test", "t_ms": 2461}
{"stage": "oracle.catalog", "msg": "reconciled against DBA_OBJECTS", "kinds": 13, "mismatches": 0, "objects": 10613, "owner": "ADEMPIERE", "cmd": "database test", "t_ms": 2635}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "database test", "t_ms": 15959}
{"stage": "run", "msg": "finished", "ok": true, "ms": 15960, "warnings": 0, "cmd": "database test", "t_ms": 15960}

What the operator gets

The step's state above.

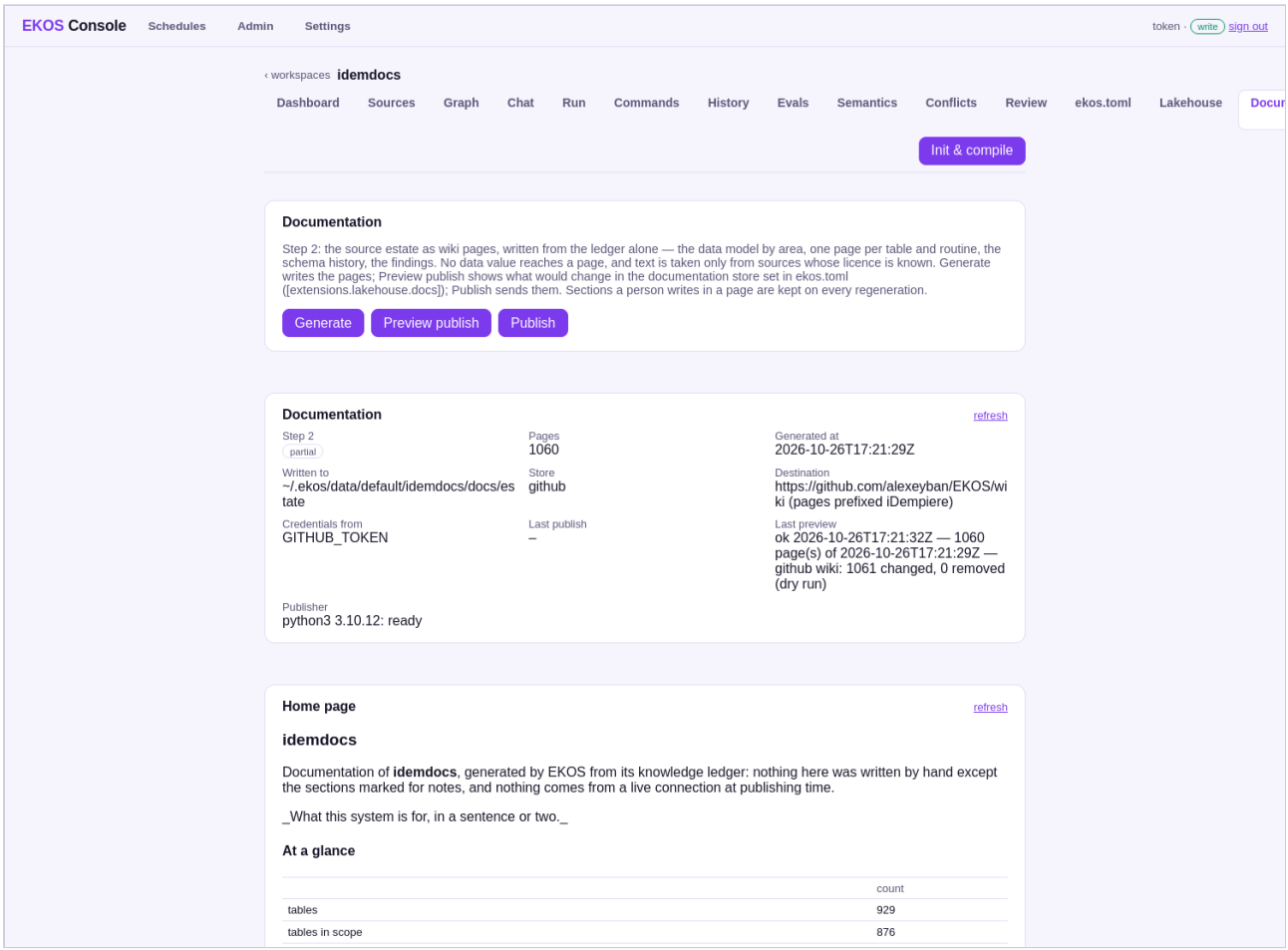
Caveats

The analysis is as good as the parsers: what a parser skipped is counted and shown, never guessed.

Step 2 — Documentation

On the step bar: partial — 1060 page(s) generated 2026-10-26T17:21:29Z into ~/.ekos/data/default/idemdocs/docs/estate; not published

What the operator sees



The Documentation page: the store, the latest generation and publish, the Home page.

What they click

On **Documentation** the operator clicks **Generate**, then **Preview publish** to see what would change in the documentation store the project names, then **Publish**.

What happens behind it

EKOS writes the source estate as wiki pages from the ledger alone: the data model by area, one page per table and routine, the history of schema changes, the findings. A profile contributes counts, never values; text from a source without a known licence is left out. Sections a person writes are kept on every regeneration. Publishing runs the

lakehouse-docs publisher for the store (GitHub wiki, Azure DevOps wiki or Confluence) with the credentials the console's environment holds; a preview sends nothing.

The run log

`lakehouse docs generate` — succeeded, from the console, 2026-10-26T17:18:51.194784; model calls: 0.


```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "lakehouse docs generate", "t_ms": 0}
{"stage": "docs", "msg": "estate pages written", "at": "2026-10-26T17:19:16Z", "committed_at": "2026-10-25 16:56 UTC", "output": "~/.ekos/data/default/idemdocs/docs/estate", "pages": 1060, "removed": 0, "unchanged": 1060, "written": 0, "cmd": "lakehouse docs generate", "t_ms": 25468}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "lakehouse docs generate", "t_ms": 26701}
{"stage": "run", "msg": "finished", "ok": true, "ms": 26701, "warnings": 0, "cmd": "lakehouse docs generate", "t_ms": 26701}
```


`lakehouse docs publish --dry-run` — succeeded, from the console, 2026-10-26T17:19:22.430122; model calls: 0.


```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "lakehouse docs publish", "t_ms": 0}
{"stage": "docs", "msg": "dry run", "ok": true, "pages": 1060, "target": "github", "cmd": "lakehouse docs publish", "t_ms": 1605}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "lakehouse docs publish", "t_ms": 1605}
{"stage": "run", "msg": "finished", "ok": true, "ms": 1605, "warnings": 0, "cmd": "lakehouse docs publish", "t_ms": 1605}
```


`lakehouse docs generate` — succeeded, from the console, 2026-10-26T17:21:03.755642; model calls: 0.


```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "lakehouse docs generate", "t_ms": 0}
{"stage": "docs", "msg": "estate pages written", "at": "2026-10-26T17:21:29Z", "committed_at": "2026-10-25 16:56 UTC", "output": "~/.ekos/data/default/idemdocs/docs/estate", "pages": 1060, "removed": 0, "unchanged": 1060, "written": 0, "cmd": "lakehouse docs generate", "t_ms": 25500}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "lakehouse docs generate", "t_ms": 26780}
{"stage": "run", "msg": "finished", "ok": true, "ms": 26780, "warnings": 0, "cmd": "lakehouse docs generate", "t_ms": 26780}
```


`lakehouse docs publish --dry-run` — succeeded, from the console, 2026-10-26T17:21:31.162141; model calls: 0.


```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "lakehouse docs publish", "t_ms": 0}
{"stage": "docs", "msg": "dry run", "ok": true, "pages": 1060, "target": "github", "cmd": "lakehouse docs publish", "t_ms": 1607}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "lakehouse docs publish", "t_ms": 1607}
{"stage": "run", "msg": "finished", "ok": true, "ms": 1607, "warnings": 0, "cmd": "lakehouse docs publish", "t_ms": 1607}
```


What the operator gets

The step's state above.

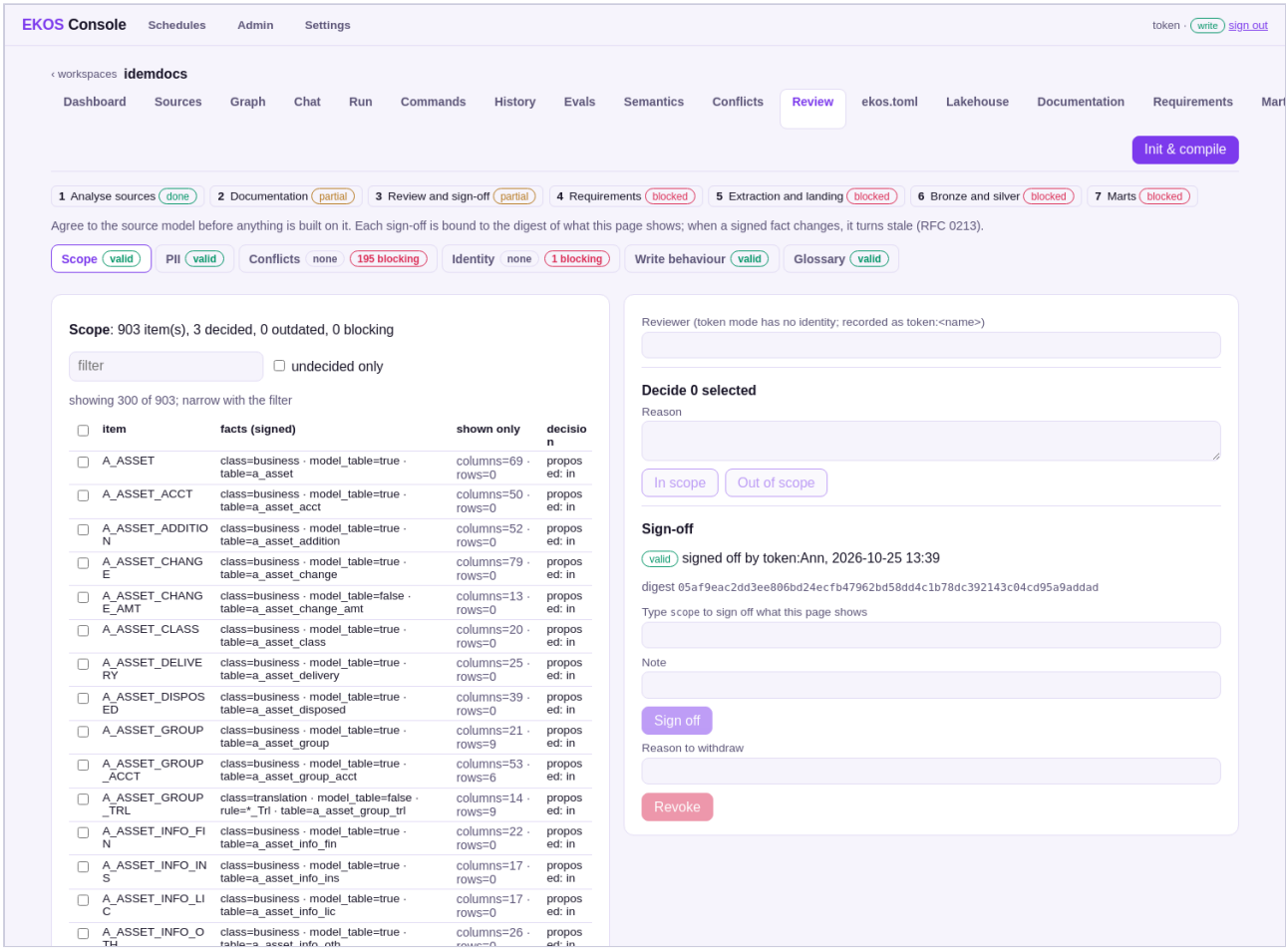
Caveats

Publishing to a public wiki is the operator's decision every time; until the latest generation is published, step 2 stays partial.

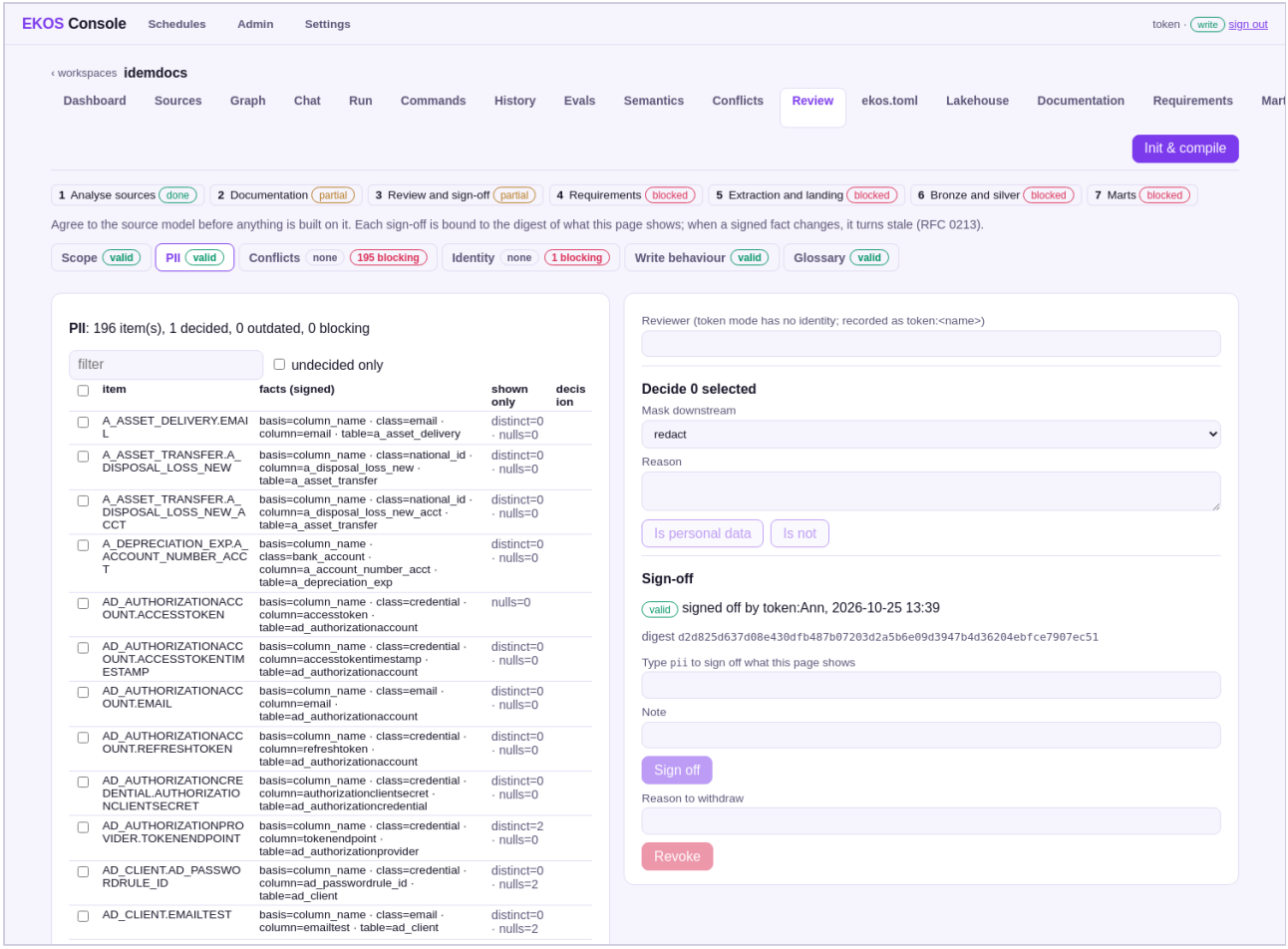
Step 3 — Review and sign-off

On the step bar: partial — 4 of 6 subjects signed, 196 blocking item(s)

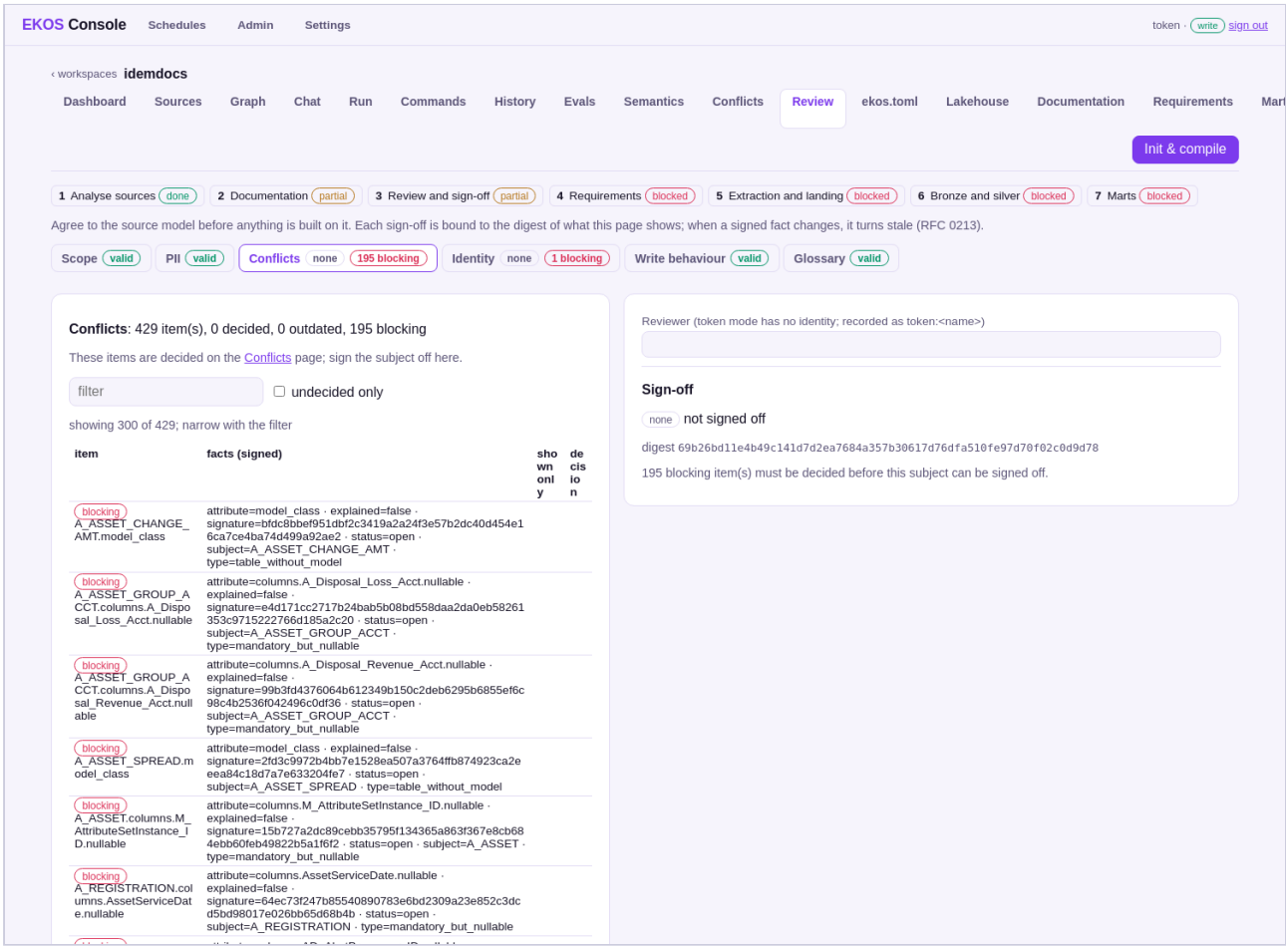
What the operator sees



Review: scope — which tables are business data.



Review: personal data, with the proposed masking.



Review: where the code and the database disagree.

What they click

Per subject (scope, PII, write behaviour, glossary, conflicts, identity) the operator decides the open items, then signs the subject off by typing its name.

What happens behind it

Every decision is a ledger fact. A sign-off is bound to a digest of exactly what was shown; when a later analysis changes one of those facts, the sign-off turns stale by itself. Steps 4–7 stay blocked until every subject is signed.

The run log

No run of this step's commands was found in the console's history or the project's run logs (`signoff`, `review`, `conflicts resolve`, `identity`).

What the operator gets

Subject	Items	Decided	Blocking	Sign-off
scope	903	3	0	valid
pii	196	1	0	valid
conflicts	429	0	195	none
identity	1	0	1	none
write-behaviour	850	0	0	valid
glossary	0	0	0	valid

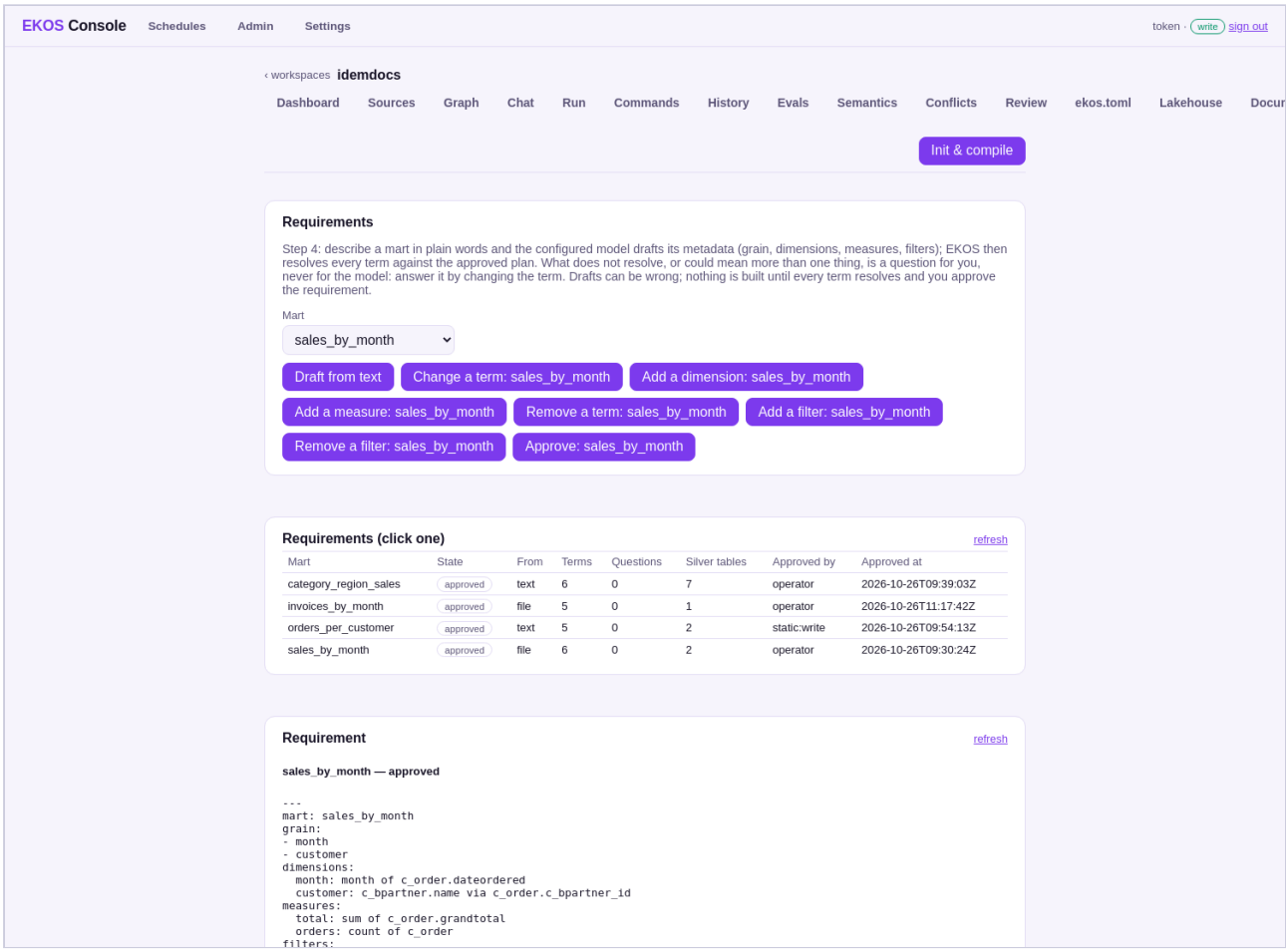
Caveats

Hundreds of conflicts take a person's time; the review groups them, but it does not decide them.

Step 4 — Requirements

On the step bar: blocked — needs a valid sign-off of every review subject (step 3); 4 of 4 requirement(s) approved

What the operator sees



Requirements: each mart, its terms and how each resolved.

What they click

The operator describes a mart in plain words (or as Markdown), answers the questions on the page by editing a term, and approves it.

What happens behind it

The configured model (local Ollama here) drafts the mart's header from the request — the only model call in the workflow. EKOS then resolves every term against the approved source model: a column, a join path along declared keys, a filter. What does not resolve becomes a question for the operator, never for the model. Approval commits the description and records who approved which digest.

The run log

`lakehouse requirements draft --mart=orders_per_customer --text=number of completed sales orders per customer and month` — succeeded, from the console, 2026-10-26T09:48:55.924491; model calls: not recorded.

This run left no structured log (it ran before EKOS RFC 0218).

`lakehouse requirements approve --mart=orders_per_customer --note=approved in the browser --as=static:write` — succeeded, from the console, 2026-10-26T09:54:13.084025; model calls: not recorded.

This run left no structured log (it ran before EKOS RFC 0218).

`lakehouse requirements edit --mart=category_region_sales --filter=c_orderline.qtyordered > 0 --as=static:write` — succeeded, from the console, 2026-10-26T14:08:10.581099; model calls: 0.


```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "lakehouse requirements edit", "t_ms": 0}
{"stage": "requirements", "msg": "resolved", "mart": "category_region_sales", "questions": 0, "reads":
["C_ORDERLINE", "C_ORDER", "C_BPARTNER_LOCATION", "C_LOCATION", "C_REGION", "M_PRODUCT",
"M_PRODUCT_CATEGORY"], "state": "changed", "terms": 7, "cmd": "lakehouse requirements edit", "t_ms": 72}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "lakehouse
requirements edit", "t_ms": 84}
{"stage": "run", "msg": "finished", "ok": true, "ms": 84, "warnings": 0, "cmd": "lakehouse requirements
edit", "t_ms": 84}
```


`lakehouse requirements edit --mart=category_region_sales --drop-filter=c_orderline.qtyordered > 0 --as=static:write` — succeeded, from the console, 2026-10-26T14:08:12.008962; model calls: 0.


```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "lakehouse requirements edit", "t_ms": 0}
{"stage": "requirements", "msg": "resolved", "mart": "category_region_sales", "questions": 0, "reads":
["C_ORDERLINE", "C_ORDER", "C_BPARTNER_LOCATION", "C_LOCATION", "C_REGION", "M_PRODUCT",
"M_PRODUCT_CATEGORY"], "state": "approved", "terms": 6, "cmd": "lakehouse requirements edit", "t_ms":
72}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "lakehouse
requirements edit", "t_ms": 87}
{"stage": "run", "msg": "finished", "ok": true, "ms": 87, "warnings": 0, "cmd": "lakehouse requirements
edit", "t_ms": 87}
```


What the operator gets

Mart	State	Terms	Questions	Described as	Approved
category_region_sales	approved	6	0	text	operator 2026-10-26T09:39:03Z
invoices_by_month	approved	5	0	file	operator 2026-10-26T11:17:42Z
orders_per_customer	approved	5	0	text	static:write 2026-10-26T09:54:13Z
sales_by_month	approved	6	0	file	operator 2026-10-26T09:30:24Z

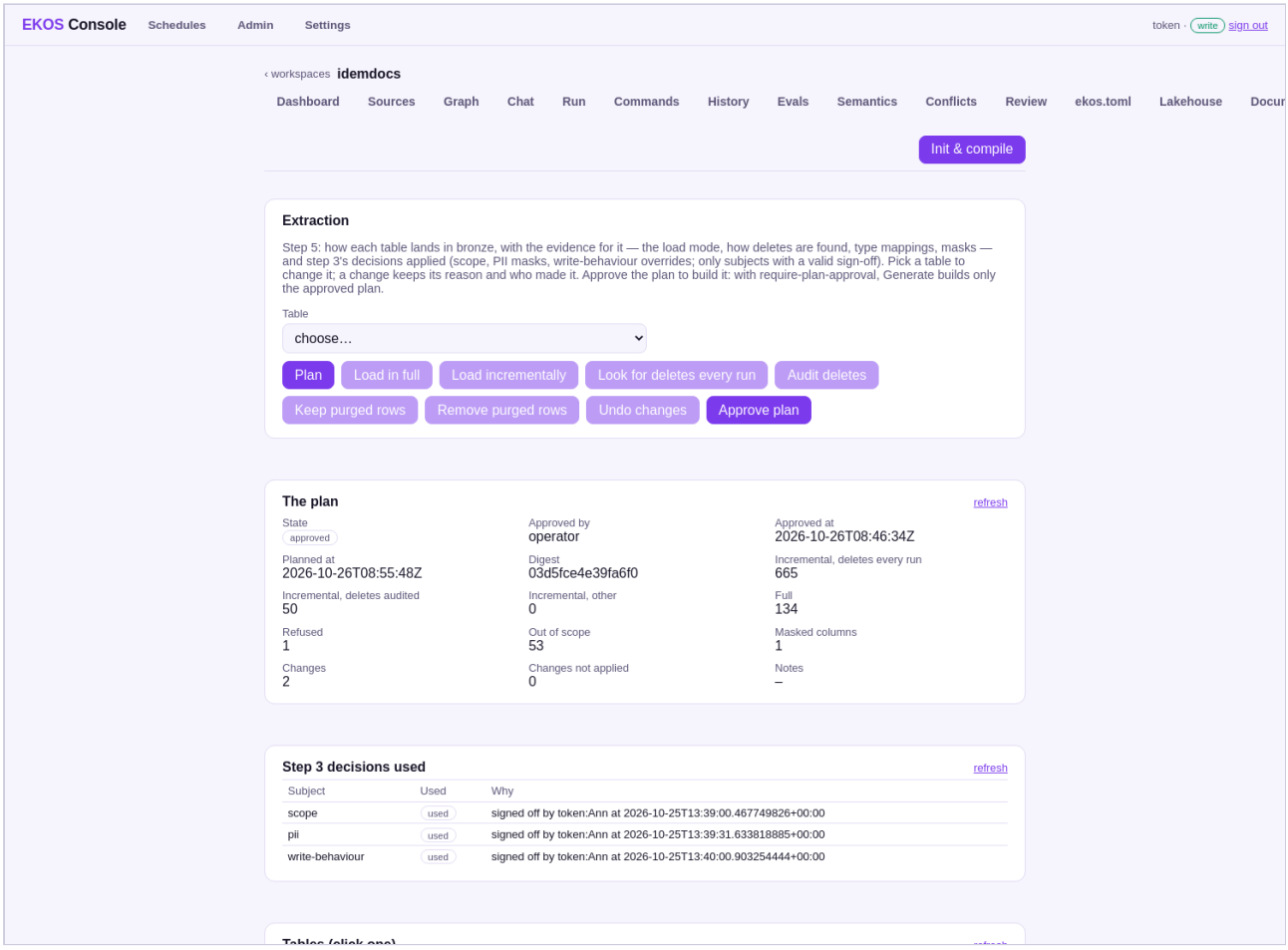
Caveats

A model can draft a term that resolves and is still wrong; that is why approval is a person's.

Step 5 — Extraction and landing

On the step bar: blocked — needs a valid sign-off of every review subject (step 3); plan approved by operator at 2026-10-26T08:46:34Z: 715 incremental, 134 full, 1 refused, 53 out of scope

What the operator sees



Extraction: per table, load mode, deletes, masking, and why.

What they click

The operator reads the proposed plan per table, changes what they disagree with (with a reason), and approves the plan.

What happens behind it

The planner proposes how each table is loaded from what step 1 found: incremental from an application watermark where the code updates it, full otherwise; how deletes are detected (bucketed key diff, with a guard against mass deletes); purged rows kept; personal data masked as step 3 decided. Approval binds the plan's digest; generating the pipeline refuses another plan.

The run log

`lakehouse extraction approve --note=C_ORDER in full for now --as=static:write` — succeeded, from the console, 2026-10-25T23:52:01.242921; model calls: not recorded.

This run left no structured log (it ran before EKOS RFC 0218).

`lakehouse extraction approve --note=C_BPARTNER in full for now --as=static:write` — succeeded, from the console, 2026-10-25T23:58:13.418732; model calls: not recorded.

This run left no structured log (it ran before EKOS RFC 0218).

`lakehouse extraction change --table=ADEMPIERE.C_BPARTNER --full --reason=shown in the demo book, then undone --as=static:write` — succeeded, from the console, 2026-10-26T14:08:15.127177; model calls: 0.


```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "lakehouse extraction change", "t_ms": 0}
{"stage": "extraction", "msg": "changed", "change": "load in full", "table": "ADEMPIERE.C_BPARTNER",
"cmd": "lakehouse extraction change", "t_ms": 87}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "lakehouse
extraction change", "t_ms": 101}
{"stage": "run", "msg": "finished", "ok": true, "ms": 101, "warnings": 0, "cmd": "lakehouse extraction
change", "t_ms": 101}
```


`lakehouse extraction change --table=ADEMPIERE.C_BPARTNER --undo --as=static:write` — succeeded, from the console, 2026-10-26T14:08:17.422396; model calls: 0.


```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "lakehouse extraction change", "t_ms": 0}
{"stage": "extraction", "msg": "changes undone", "removed": 1, "table": "ADEMPIERE.C_BPARTNER", "cmd":
"lakehouse extraction change", "t_ms": 53}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "lakehouse
extraction change", "t_ms": 64}
{"stage": "run", "msg": "finished", "ok": true, "ms": 64, "warnings": 0, "cmd": "lakehouse extraction
change", "t_ms": 64}
```


What the operator gets

Tables	Count
incremental, deletes every run	665
incremental, deletes audited	50
incremental, other	0
full	134
refused (type cannot carry the data)	1
out of scope (step 3)	53
masked columns	1
operator changes	2

Plan `03d5fce4e39fa6f0`, approved by operator at 2026-10-26T08:46:34Z.

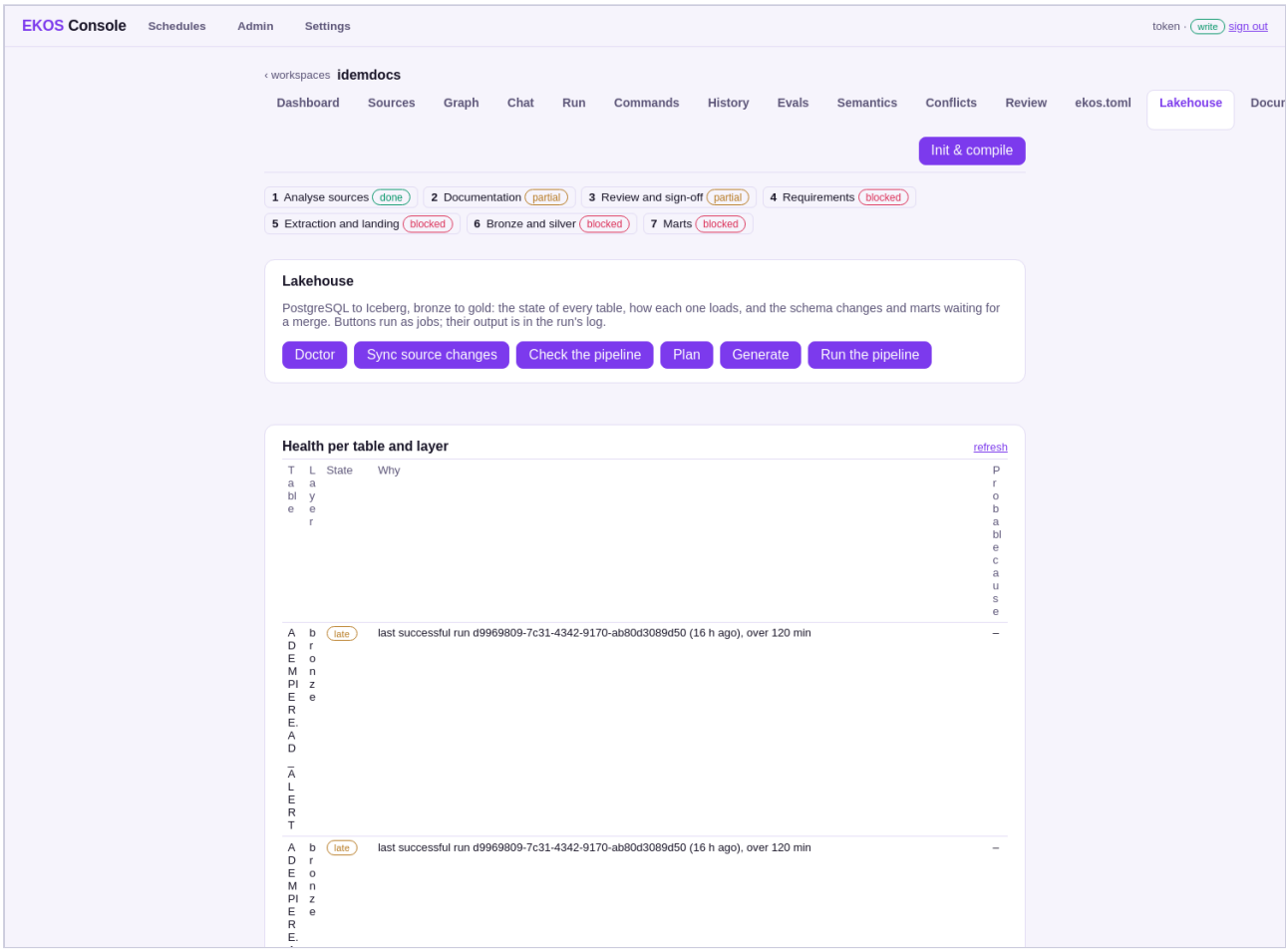
Caveats

Tables whose data a type cannot carry exactly are refused and named, never rounded.

Step 6 — Bronze and silver

On the step bar: blocked — needs a valid sign-off of every review subject (step 3); 325 bronze/silver table(s) built, 142 failing

What the operator sees



Lakehouse: each table's latest load and its state.

What they click

The operator generates the pipeline and starts the load.

What happens behind it

EKOS generates the extraction SQL, the dbt-trino silver models with their tests (keys, check lists, code lists the application declares) and the Dagster job. Dagster loads bronze into Iceberg and builds silver; the run's results come back to the ledger per table.

The run log

No run of this step's commands was found in the console's history or the project's run logs (`lakehouse run`, `lakehouse generate`, `lakehouse sync`, `lakehouse run-report`, `lakehouse reconcile`).

What the operator gets

Layer	State	Tables
bronze	late	849
gold	late	4
silver	failing	142
silver	late	23

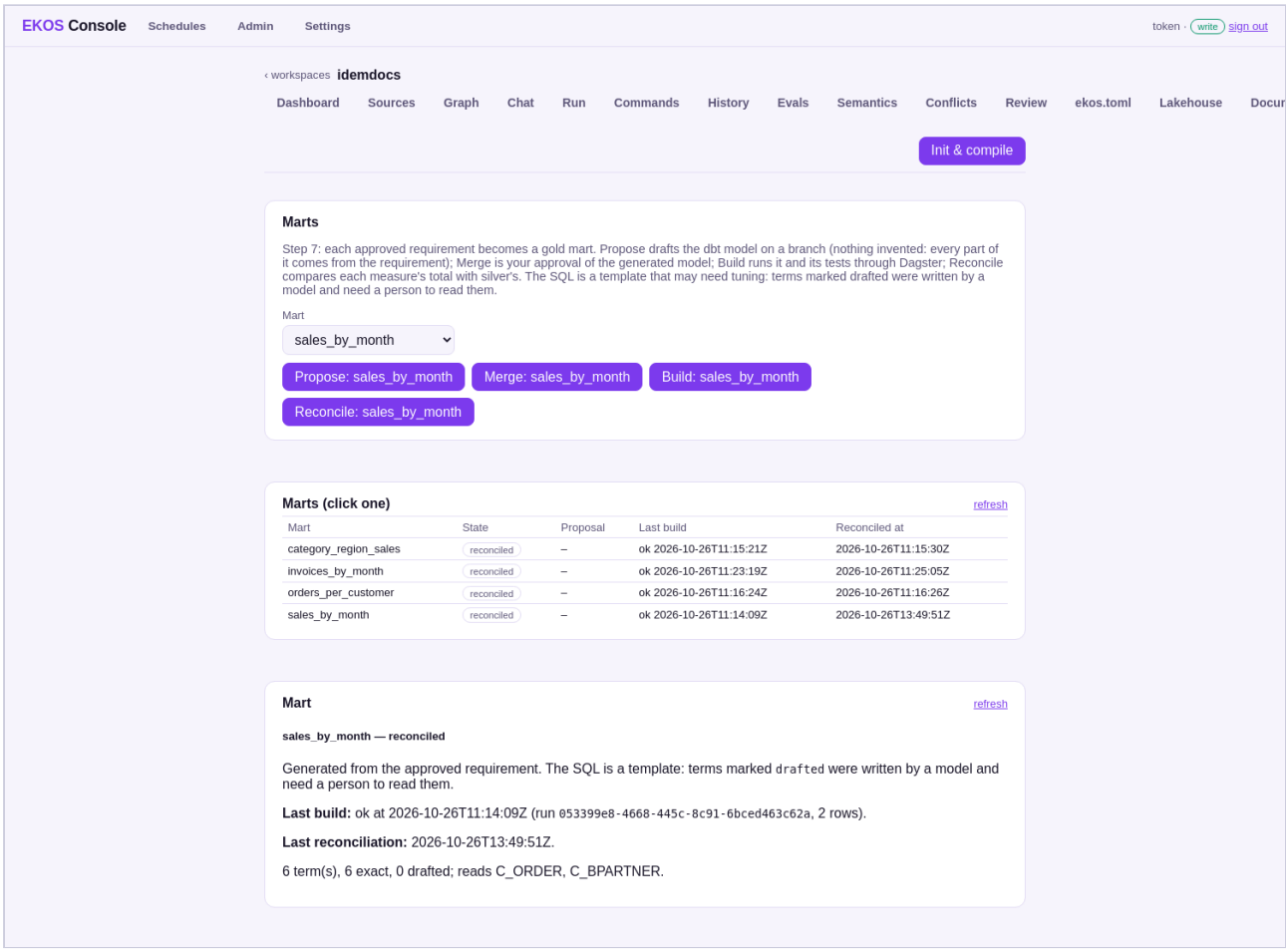
Caveats

On this laptop testbed a 165-table subset was loaded, not all 849 tables (EKOS RFC 0227).

Step 7 — Marts

On the step bar: blocked — needs a valid sign-off of every review subject (step 3); 4 of 4 mart(s) reconciled

What the operator sees



Marts: a mart's SQL, terms, tests, totals and lineage.

What they click

Per approved requirement: **Propose**, **Merge** (the approval of the generated model), **Build**, **Reconcile**.

What happens behind it

Propose compiles the approved description into a dbt gold model on a branch and checks that nothing in it was invented. Merging is the approval. Build runs only that mart and its tests through Dagster. Reconcile reads each measure's total from the mart and the same total computed from silver, and shows both.

The run log

`lakehouse marts build --mart=invoices_by_month` — failed, from the console, 2026-10-26T11:18:08.172534; model calls: not recorded.

This run left no structured log (it ran before EKOS RFC 0218).

`lakehouse marts build --mart=invoices_by_month` — succeeded, from the console, 2026-10-26T11:21:36.828306; model calls: not recorded.

This run left no structured log (it ran before EKOS RFC 0218).

`lakehouse marts reconcile --mart=invoices_by_month` — succeeded, from the console, 2026-10-26T11:25:04.857142; model calls: not recorded.

This run left no structured log (it ran before EKOS RFC 0218).

`lakehouse marts reconcile --mart=sales_by_month` — succeeded, from the console, 2026-10-26T13:49:49.538054; model calls: 0.


```
{"stage": "run", "msg": "started", "version": "1.1.0", "cmd": "lakehouse marts reconcile", "t_ms": 0}
{"stage": "marts", "msg": "reconciled", "equal": true, "mart": "sales_by_month", "mart_total": "379.90",
"measure": "total", "silver_total": "379.90", "cmd": "lakehouse marts reconcile", "t_ms": 3035}
{"stage": "marts", "msg": "reconciled", "equal": true, "mart": "sales_by_month", "mart_total": "3",
"measure": "orders", "silver_total": "3", "cmd": "lakehouse marts reconcile", "t_ms": 3035}
{"stage": "llm", "msg": "calls", "count": 0, "cache_hits": 0, "embeddings": 0, "cmd": "lakehouse marts
reconcile", "t_ms": 3553}
{"stage": "run", "msg": "finished", "ok": true, "ms": 3553, "warnings": 0, "cmd": "lakehouse marts
reconcile", "t_ms": 3553}
```


What the operator gets

Mart	State	Last build
category_region_sales	reconciled	ok 2026-10-26T11:15:21Z
invoices_by_month	reconciled	ok 2026-10-26T11:23:19Z
orders_per_customer	reconciled	ok 2026-10-26T11:16:24Z
sales_by_month	reconciled	ok 2026-10-26T11:14:09Z

Each measure's total in the mart and the same total computed from silver:

Mart	Measure	Mart total	Silver total	
category_region_sales	sales	358.40	358.40	equal
invoices_by_month	invoiced	541.02	541.02	equal
invoices_by_month	invoices	4	4	equal
orders_per_customer	count_of_c_order_id	7	7	equal
sales_by_month	total	379.90	379.90	equal
sales_by_month	orders	3	3	equal

Caveats

Generated transformations are a template that may need tuning; terms a model drafted are marked.